

Reading and writing IAM policies: conditions and condition keys

The elements, the operators, and the four traps that make a correct-looking policy do the opposite.

Most broken policies are not wrong. They are **correct about something else.**

The elements, quickly

- ◆ **Effect** Allow or Deny. **Action** the API calls. **Resource** the ARNs. **Condition** when it applies.
- ◆ **Principal** appears in **resource-based policies and trust policies only**. An identity policy never has one – the identity is implied.
- ◆ **Sid** is a label. It has no effect on evaluation and is invaluable when reading CloudTrail or a simulator result.
- ◆ A statement matches only if **every** element matches – action, resource and every condition.
- ◆ SCS-C03 removed these as a stated skill. You still cannot read a policy without them.

ON THE EXAM

A Principal element in something described as an identity policy is a sign the question is describing a resource policy.

NotAction, NotResource and the wildcard problem

- ◆ **Allow with NotAction grants every action you did not list** – across every service, including services that do not exist yet.
- ◆ AWS guidance: prefer an **explicit Deny with Action** over an Allow with NotAction. It expresses the same intent and cannot grow.
- ◆ **Deny with NotAction** is the useful form – deny everything except this small set. That is how allow-list SCPs are written.
- ◆ **NotPrincipal** is genuinely hard to reason about and AWS advises against it in most cases.
- ◆ **s3:*** includes s3:PutBucketPolicy and s3>DeleteBucket. A wildcard on a service is an administrative grant.

ON THE EXAM

Read every Not- element twice. The set they invert grows every AWS launch.

Condition operators worth knowing

OPERATOR	USE
StringEquals / StringLike	Exact match; StringLike allows * and ?
ArnEquals / ArnLike	Match ARNs, with wildcards in ArnLike
Bool	true or false – MFA, SecureTransport
IpAddress / NotIpAddress	CIDR ranges
DateLessThan / DateGreaterThan	Time-bounded access
NumericLessThan / GreaterThan	MultiFactorAuthAge, object size
Null	Test whether a key is present at all

ON THE EXAM

Every operator has an ...IfExists variant, and that suffix changes the answer more often than the operator does.

The IfExists suffix

- ◆ **Bool** matches only when the key is present **and** the value matches.
- ◆ **BoolIfExists** matches when the key is present and matches, **or when the key is not there at all**.
- ◆ So IfExists in an **Allow** makes it more permissive – it lets through requests where the key cannot exist.
- ◆ And IfExists in a **Deny** makes it stricter – it also denies requests where the key cannot exist.
- ◆ That inversion is why the same suffix is the right answer to opposite-sounding requirements.

ON THE EXAM

IfExists does not mean 'only if present'. It means 'or if absent', which is the opposite of how it reads.

Trap one: requiring MFA

- ◆ **aws:MultiFactorAuthPresent** is absent entirely for long-term access keys. Present only for temporary credentials.
- ◆ So **Deny with Bool false** never matches a CLI call made with access keys.
- ◆ **Deny with BoolIfExists false** denies non-MFA temporary credentials **and** long-term key requests. Stricter; breaks scripts.
- ◆ **Allow with BoolIfExists true** is the console's 'require MFA' option – MFA for console, none needed for CLI.
- ◆ Console users are on temporary credentials, so the key **is** present and false for them.

ON THE EXAM

Two policies that both read as 'require MFA', behaving completely differently.

Trap two: the two source IP keys

- ◆ `aws:SourceIp` is present only when the request did not go through a VPC endpoint.
- ◆ `aws:VpcSourceIp` only when it did. The two never appear in the same request.
- ◆ So denying unless `aws:SourceIp` is in your office range **blocks everything behind a VPC endpoint**.
- ◆ Use `aws:SourceVpc` or `aws:SourceVpce` for 'from my VPC', alongside `aws:SourceIp`.
- ◆ Add `aws:ViaAWSService` so a service calling on your behalf is not caught.

ON THE EXAM

The classic data perimeter outage: correct when written, then VPC endpoints arrived.

Trap three: ForAllValues and ForAnyValue

- ◆ These set operators are for **multivalued** keys such as `aws:TagKeys`, where the request carries a list.
- ◆ **ForAnyValue** matches if **at least one** value in the request is in your list.
- ◆ **ForAllValues** matches if **every** value in the request is in your list.
- ◆ **And ForAllValues matches when the request carries no values at all**, because 'every' is vacuously true over an empty set.
- ◆ So **ForAllValues can never be used to require** that something is present. It restricts what may appear, not what must.

ON THE EXAM

ForAllValues is an allow-list on what is permitted, not a requirement that anything is supplied. Use `Null` to require presence.

Global condition keys worth knowing cold

KEY	WHAT IT CONSTRAINS
<code>aws:PrincipalOrgID</code>	Caller is in your organisation
<code>aws:PrincipalArn</code> / <code>PrincipalAccount</code>	Exactly who is calling
<code>aws:ResourceOrgID</code> / <code>ResourceAccount</code>	The resource belongs to you
<code>aws:SourceArn</code> / <code>aws:SourceAccount</code>	Confused deputy defence for service principals
<code>aws:SourceVpc</code> / <code>aws:SourceVpce</code>	Request arrived via your VPC or endpoint
<code>aws:SecureTransport</code>	TLS was used
<code>aws:RequestedRegion</code>	Which Region the call targets
<code>aws:PrincipalTag</code> / <code>RequestTag</code> / <code>ResourceTag</code>	ABAC – module 24

ON THE EXAM

PrincipalOrgID and ResourceOrgID are the two halves of a data perimeter: my people, and my resources.

Policy variables

- ◆ `${aws:username}` and `${aws:userid}` – one policy, a home prefix per person.
- ◆ `${aws:PrincipalTag/team}` substitutes a tag on the caller. This is how ABAC scales.
- ◆ Same idea as the IoT thing policy variables in module 13.
- ◆ They work in **Resource** and in condition values. **Not in Action.**
- ◆ A missing tag means the variable does not resolve and the statement does not match.

ON THE EXAM

A missing tag fails closed. Correct behaviour, and confusing to debug.

A company attaches this statement to all developer roles, intending to force MFA for destructive actions: Effect Deny, Action ec2:TerminateInstances and s3:DeleteBucket, Resource *, Condition Bool aws:MultiFactorAuthPresent false.

Console users are correctly blocked without MFA. But a developer using long-term access keys from a laptop script successfully terminates instances.

Why, and what is the fix?

- ☐ A The key is absent for long-term credentials, so Bool does not match; use BoolIfExists
- ☐ B Deny statements do not apply to programmatic access; add a matching SCP
- ☐ C The Resource element must list specific ARNs for a Deny to take effect
- ☐ D The developer's access keys were created before the policy; rotate them

◆ PAUSE AND DECIDE

Why, and what is the fix?

- A

 Key absent for long-term credentials; use BoolIfExists
Correct. The key is only present for temporary credentials, so with plain Bool the condition never matches and the Deny never fires for an access-key call.
- B

 Deny does not apply to programmatic access
False, and it contradicts module 19's absolute rule. An explicit Deny applies to every request path; this one simply never matched.
- C

 Resource must list specific ARNs
*Resource * matches everything, which is exactly what a broad Deny wants. Narrowing it would make the statement weaker, not stronger.*
- D

 Rotate the pre-existing access keys
Policies apply at evaluation time, not at credential creation time. A newly created key would behave identically.

THE SIGNAL

Console users blocked, script users not, is the signature. Console sessions are temporary credentials; access keys are not.

A bank deploys an SCP denying all S3 actions unless `aws:SourceIp` is within its two office CIDR ranges. It works correctly for six months.

The platform team then migrates workloads onto S3 gateway VPC endpoints for cost and security reasons. Immediately, every application that reaches S3 through an endpoint receives `AccessDenied`, while office users are unaffected.

What is happening, and what should the bank do?

- ☐ A Gateway endpoints do not support IAM conditions; switch to interface endpoints
- ☐ B The endpoint policy is overriding the SCP; widen the endpoint policy
- ☐ C `aws:SourceIp` is absent for requests via a VPC endpoint; also allow `aws:SourceVpce` and `aws:ViaAWSService`
- ☐ D The SCP must be reattached after any VPC change to pick up the new routes

◆ PAUSE AND DECIDE

What is happening, and what should the bank do?

A Gateway endpoints do not support IAM conditions
Fabricated. Gateway endpoints work with IAM conditions perfectly well – they just populate a different set of context keys.

B The endpoint policy overrides the SCP
Nothing overrides an SCP. It is a ceiling, and module 19's rule is that ceilings only ever subtract.

C aws:SourceIp is absent via an endpoint; add aws:SourceVpce and aws:ViaAWSService
Correct. The two source-IP keys are mutually exclusive. Requests via an endpoint carry aws:VpcSourceIp instead, so the office condition cannot be satisfied.

D Reattach the SCP after VPC changes
Invented maintenance. SCPs evaluate at request time against current context; there is nothing cached to refresh.

THE SIGNAL

The policy did not change and the traffic path did. Any IP-based condition is really a condition about the path.

In 2023 a company wrote an identity policy for its platform team: Effect Allow, NotAction iam:* and organizations:*, Resource *. The intent was 'everything except identity and organisation management'.

A 2026 audit finds the platform team can use several AWS services launched since 2023, including services that manage encryption keys and data pipelines, which were never reviewed or approved.

What should the company do?

- ☐ A Add the new services to the NotAction list and repeat this at each AWS launch
- ☐ B Add a Deny statement for the new services alongside the existing Allow
- ☐ C Attach a permissions boundary listing the new services
- ☐ D Replace the NotAction Allow with an explicit list of allowed actions, plus a Deny for iam and organizations

◆ PAUSE AND DECIDE

What should the company do?

- A

 Keep adding to the NotAction list
Signs you up to track every AWS launch forever, and the grant is live from the moment a service ships until somebody notices.
- B

 Add a Deny for the new services
Closes today's gap and leaves the mechanism intact. The next service AWS launches is granted again on day one.
- C

 A permissions boundary listing the new services
A boundary caps rather than grants, so listing services in it would permit them. To help, the boundary would need the allow-list this option avoids writing.
- D

 Explicit Allow list, plus a Deny for iam and organizations
Correct. An enumerated Allow cannot grow when AWS ships something, and the Deny expresses the original intent in the form AWS recommends.

THE SIGNAL

An Allow with NotAction is a standing grant on every service AWS has not built yet. The set it inverts keeps growing.

A company requires every EC2 instance to be created with a CostCentre tag. An engineer writes a Deny on `ec2:RunInstances` with Condition `ForAllValues:StringNotEquals` on `aws:TagKeys` with value `CostCentre`.

Testing shows the policy correctly blocks a request tagged `Owner=alice`. But a request with no tags at all is allowed, and the instance is created untagged.

Why, and how should the requirement be expressed?

- A `aws:TagKeys` is not supported for `ec2:RunInstances`; use `aws:RequestTag` instead
- B `ForAllValues` is vacuously true over an empty set; use `Null` on `aws:RequestTag/CostCentre` to require presence
- C The condition should use `ForAnyValue:StringNotEquals` instead
- D Deny statements cannot evaluate tags at creation time; use a Config rule with remediation

◆ PAUSE AND DECIDE

Why, and how should the requirement be expressed?

- A

`aws:TagKeys` unsupported for `RunInstances`
It is supported. The key is populated; the request simply carries an empty list of tag keys, which is what the set operator then evaluates.
- B

`ForAllValues` is vacuously true over an empty set; use `Null`
Correct. Set operators constrain which values may appear, never that any appear. Null tests presence, which is what 'must be tagged' actually means.
- C

 Use `ForAnyValue` instead
Changes which values trigger the match and not the empty-set behaviour. An untagged request still supplies nothing for either operator to match.
- D

 Deny cannot evaluate tags at creation
False - `aws:RequestTag` and `aws:TagKeys` exist precisely to evaluate tags on the creating request. Config would also be corrective rather than preventive.

THE SIGNAL

Set operators answer 'which values are permitted'. Only `Null` answers 'was anything supplied at all'.

Five things to carry forward

- ◆ **IfExists** means 'or if absent', not 'only if present'. It loosens an Allow and tightens a Deny.
- ◆ **aws:MultiFactorAuthPresent** is absent for long-term keys. Plain Bool never fires for a CLI script.
- ◆ **aws:SourceIp** and **aws:VpcSourceIp** are mutually exclusive. An IP condition is really a condition about the path.
- ◆ **ForAllValues** is true over an empty set. Use Null to require that something is supplied.
- ◆ **Allow with NotAction** grants services that do not exist yet. Prefer an enumerated Allow plus an explicit Deny.

ON THE EXAM

Next: roles, trust policies, cross-account access and the confused deputy.